

Atividade Prática 1: Implementação do Algoritmo K-Means em Python

Objetivo

Com o objetivo principal de exercitar nossas habilidades em Python, vamos compreender o funcionamento do algoritmo de clusterização *K*-Means, analisar sua estrutura lógica e implementar a solução em Python nativo (sem o uso de bibliotecas de Machine Learning ou uso de ferramentas de IA generativa).

1 O Algoritmo K-Means

O *K*-Means é um dos algoritmos de aprendizado não supervisionado mais populares para a tarefa de **clusterização**. Seu objetivo é particionar um conjunto de N dados em K grupos distintos, onde cada ponto pertence ao grupo com o centroide mais próximo.

Funcionamento Passo a Passo

1. **Definição de K :** Escolhe-se o número K de grupos desejados.
2. **Inicialização dos Centroides:** Selecionam-se aleatoriamente K pontos do conjunto de dados para servirem como centroides iniciais.
3. **Atribuição aos Clusters:** Para cada ponto do dataset, calcula-se a distância (usaremos a *Distância Euclidiana*) em relação a cada um dos K centroides. O ponto é atribuído ao cluster do centroide mais próximo.
4. **Atualização dos Centroides:** Para cada cluster, recalcula-se a posição do centroide tomando a média aritmética das coordenadas de todos os pontos atribuídos a ele.
5. **Convergência:** Os passos 3 e 4 são repetidos até que ocorra um critério de parada:
 - Os centroides não alteram mais suas posições ou a mudança dos mesmos é menor que um limiar ϵ .
 - O número máximo de iterações pré-definido é atingido.

Fórmulas de Apoio

- **Distância Euclidiana** entre dois pontos $P = (p_1, p_2, \dots, p_n)$ e $Q = (q_1, q_2, \dots, q_n)$:

$$d(P, Q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

- **Novo Centroide** C_k de um grupo com M pontos:

$$C_k = \left(\frac{1}{M} \sum x_1, \frac{1}{M} \sum x_2, \dots, \frac{1}{M} \sum x_n \right)$$

2 Pseudocódigo

```
ALGORITMO KMeans(Dados, K, MaxIteracoes)

// Inicializacao
Centroides = Selecionar K pontos aleatorios de Dados
Iteracao = 0

ENQUANTO Iteracao < MaxIteracoes E CentroidesMudaram FACA:

    // Inicializar K clusters vazios
    Clusters = Criar K listas vazias

    // Atribuicao
    PARA CADA Ponto EM Dados FACA:
        MenorDistancia = INFINITO
        IndiceCluster = -1

        PARA i DE 0 A K-1 FACA:
            Dist = CalcularDistanciaEuclidiana(Ponto,
Centroides[i])
            SE Dist < MenorDistancia ENTAO:
                MenorDistancia = Dist
                IndiceCluster = i
            FIM SE
        FIM PARA

        Adicionar Ponto ao Clusters[IndiceCluster]
    FIM PARA

    // Atualizacao dos Centroides
    NovosCentroides = Lista de tamanho K
    PARA i DE 0 A K-1 FACA:
        SE Clusters[i] NAO ESTA VAZIO ENTAO:
            NovosCentroides[i] = CalcularMediaDosPontos(
Clusters[i])
        SENAO:
```

```

        NovosCentroides[i] = Centroides[i] // Mantem se
ficar sem pontos
        FIM SE
    FIM PARA

    // Verificar Parada
    SE NovosCentroides == Centroides ENTAO:
        CentroidesMudaram = FALSO
    SENA0:
        Centroides = NovosCentroides
        Iteracao = Iteracao + 1
    FIM SE

FIM ENQUANTO

RETORNAR Clusters, Centroides
FIM ALGORITMO

```

3 Instruções para a Implementação

Regras da Tarefa

- **Proibido o uso de bibliotecas de terceiros** como `scikit-learn`, `numpy`, `pandas` ou `scipy`. Utilize apenas recursos nativos da linguagem Python (listas, dicionários, módulo `math`, `random`, `csv`, etc.).
- **Proibido o uso de ferramentas de IA generativa** para escrita de código. O objetivo é exercitar a lógica e as estruturas da linguagem.
- Seu programa deve ler um dos arquivos de dados fornecidos (formato CSV), executar o *K*-Means e exibir os centroides finais e os pontos atribuídos a cada cluster.

Níveis de Implementação

- **Iniciante/Intermediário:** Implementação utilizando funções estruturadas e tipos nativos de dados (listas/dicionários).
- **Avançado (Recomendado para quem já domina a sintaxe):** Utilize **Orientação a Objetos (POO)**.
 - *Sugestão de classes:* `Ponto`, `Cluster` e `KMeansAlgorithm`.

4 Arquivos de Teste (Datasets)

Abaixo estão as estruturas de 4 conjuntos de dados em formato CSV para serem criados e salvos em arquivos `.csv` na mesma pasta do script Python.

Arquivo 1: dados_1_simple.csv

Dados simples com 2 grupos bem separados, ideal para debug inicial.

```
x,y
1.0,2.0
1.5,1.8
5.0,8.0
8.0,8.0
1.2,0.8
9.0,11.0
8.0,9.0
1.0,0.6
2.0,0.9
9.0,10.0
```

Arquivo 2: dados_2_3clusters.csv

Dados 2D com 3 grupos bem definidos — teste de $K = 3$.

```
x,y
10.0,10.0
11.0,12.0
9.5,11.5
10.5,9.0
50.0,50.0
52.0,48.0
49.0,51.0
51.5,49.5
90.0,10.0
92.0,11.0
89.0,9.0
91.0,12.0
```

Arquivo 3: dados_3_3d.csv

Dados em 3 dimensões X, Y, Z com $K = 2$ — para testar funcionamento em mais dimensões.

```
x,y,z
1.0,1.0,1.0
1.5,2.0,1.2
1.1,1.3,0.9
10.0,10.0,10.0
10.5,9.8,10.2
9.9,10.1,9.7
1.2,1.1,1.4
10.2,10.3,10.1
```

5 Entregável

Entregar o código comentado e um arquivo com um breve relatório dos testes realizados e a dificuldades encontradas.